

Full Stack Design (FSD) Project Documentation: ShopEZ Stock Trader

Parameter	Details
Date	18 June 2026
Team ID	SE-09
Project Name	ShopEZ Stock Trading Platform
Maximum Marks	4 Marks

1. Introduction

- **Project Title:** ShopEZ Stock Trading Platform (MERN Stack Real-Time Paper-Trading Sandbox)
- **Team Members & Roles:** SE-09 Operations Team

2. Project Overview

Purpose of the System

The ShopEZ Stock Trading Platform addresses the gap in virtual paper-trading environments for beginners. By providing a clean light/pastel website where users can search stock listings, view live price simulations, and execute buy/sell orders, it digitizes financial sandboxes and eliminates financial risk for learning investors.

Key Features

- **Role-Based Workflows:** Custom dashboards and UI views tailored for Traders and Administrators.
- **Live Price Simulation:** Node background interval script automatically updates database ticker prices to mimic tick-by-tick fluctuations.
- **Simplified Trade Engine:** Instant validation of Buy/Sell orders against virtual cash balance and share holdings.
- **Portfolio Statistics:** Automatic tracking of average purchase cost, net worth, and live profit/loss returns.
- **Admin Controls:** Panel monitoring user counts, total trades, with forms to add, update, or delete stock tickers.

3. Architecture

The platform follows a classic decoupled MERN (MongoDB, Express, React, Node.js) architecture:

Frontend Architecture

- Built on **React** (using **Vite** as compiler and bundler).
- **React Router** manages client-side page routing (protected paths for dashboards).
- **Context API (AuthContext)** stores active login tokens, current user objects, and handles session state.
- Styling implemented using premium CSS design tokens for light/pastel variables.
- **Chart.js** renders live price histories for individual stock detail pages.

Backend Architecture

- Built on **Node.js** running an **Express.js** API server.
- Handles API endpoints, input body parsing, seeder initialization, and error handling.
- Restricts actions using custom authorization middlewares: ``protect`` (verifies JWT headers) and ``authorize`` (checks roles).

MongoDB Database Architecture

- Hosted on **MongoDB Atlas** cloud database, communicating via **Mongoose ODM**.
- Consists of four core collections:
- **User**: Base identity schema (Trader, Admin), storing password hash and virtual balance.
- **Stock**: Active stock details, current price, starting price, and historical ticks array.
- **Transaction**: Trade logs showing user reference, ticker, type (BUY/SELL), shares, and execution price.
- **Portfolio** (or holding schema): Maps trader reference, ticker, shares owned, and average purchase cost.

4. Setup Instructions

Prerequisites

- **Node.js** (v18.x or higher recommended)
- **npm** (v9.x or higher)
- **MongoDB** (Local Community Edition running on port 27017 or a MongoDB Atlas cloud URI)

Installation Guide

1. Clone the Repository:

```
```bash
```

```
git clone
```

```
cd "shopez-stock-trader"
```

```
```
```

```
#### 2. Install Dependencies:
```

Install backend and frontend packages from the monorepo root:

```
```bash
```

```
npm run install-all
```

```
```
```

```
#### 3. Configure Environment Variables:
```

Create a `.env`` file under the ``/backend`` directory with the following contents:

```
```text
```

```
PORT=5000
```

```
MONGO_URI=mongodb+srv://shopez_admin:Srinithya%409838@cluster0.qhawxph.mongodb.net/shopez?retryWrites=true&w;=majority&appName;=Cluster0
```

```
JWT_SECRET=shopez_secret_key_2026_jwt_token_auth
```

```
STOCK_API_KEY=placeholder_finnhub_or_alphavantage_key
```

```
```
```

Default Seeding Accounts

At database startup, a seeding utility runs to insert initial default accounts if the database is clean:

- **Admin:** ``admin@shopez.com` / `password123``
- **Trader User:** ``user@shopez.com` / `password123``

5. Folder Structure

The repository is structured as a monorepo splitting client and server concerns:

```
``text
```

```
"shopez-stock-trader"/
```

```
■■■ frontend/ # React Frontend Single Page Application
```

```
■ ■■■ public/ # Static public assets
```

```
■ ■■■ src/
```

```
■ ■ ■■■ assets/ # App logos and design components
```

```
■ ■ ■■■ components/ # Reusable UI components (Navbar, ProtectedRoute)
```

```
■ ■ ■■■ context/ # React Context objects (AuthContext)
```

```
■ ■ ■■■ pages/ # Layout Pages (Dashboard, Market, StockDetail, Portfolio, Login, Register)
```

```
■ ■ ■■■ App.css # Main App layouts styling
```

```
■ ■ ■■■ App.jsx # App routes and page definitions
```

```
■ ■ ■■■ main.jsx # App entry mounting point
```

```
■ ■■■ package.json
```

```
■ ■■■ vite.config.js
```

```
■
```

```
■■■ backend/ # Express Backend REST API
```

■ ■■■■ config/ # Database connection and DB seeder files

■ ■■■■ controllers/ # Route controllers matching API logic

■ ■■■■ middleware/ # Security guards and auth checks

■ ■■■■ models/ # Mongoose schemas definition

■ ■■■■ routes/ # REST Express router maps

■ ■■■■ utils/ # Token generation and price simulator helpers

■ ■■■■ package.json

■ ■■■■ server.js # Server application entry configuration

...

6. Running the Application

Running Locally (Development Mode)

To launch both client and server development engines:

1. Start Server:

```
```bash
```

```
cd backend
```

```
npm run dev
```

**Runs backend server on <http://localhost:5000>**

...

#### 2. Start Client:

```
```bash
```

```
cd frontend
```

```
npm run dev
```

Runs frontend development server on <http://localhost:5173>

```
```
```

### Building for Production

To build the client SPA and start the static Node delivery server:

```
```bash
```

```
npm run build
```

```
npm start
```

```
```
```

---

## 7. API Documentation

### 7.1 Authentication Routes (`/api/auth`)

#### 1. Register User

- **HTTP Method:** POST
- **Path:** `/api/auth/register`
- **Role Authorized:** Public
- **Request Body:**

```
```json
```

```
"name": "Trader One",
```

```
"email": "trader@shopez.com",
```

```
"password": "password123",
```

```
"role": "Trader"
```

```
```
```

- **Response (201 Created):**

```
```json
```

```
"success": true,
```

```
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
```
```

## #### 2. Login User

- **HTTP Method:** POST

- **Path:** `/api/auth/login`

- **Role Authorized:** Public

- **Request Body:**

```
```json
```

```
"email": "trader@shopez.com",
```

"password": "password123"

...

- **Response (200 OK):**

```json

"success": true,

"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

...

#### #### 3. Get Current User Details

- **HTTP Method:** GET

- **Path:** `/api/auth/me`

- **Role Authorized:** Protected (Trader/Admin)

- **Response (200 OK):**

```json

"success": true,

"data":

"_id": "603d2b2f56b2cf00155b5f5a",

"name": "Trader One",

"email": "trader@shopez.com",

"role": "Trader",

"balance": 100000

...

7.2 Stock Routes (`/api/stocks`)

1. List All Stocks

- **HTTP Method:** GET
- **Path:** `/api/stocks`
- **Role Authorized:** Public
- **Response (200 OK):**

``json

"success": true,

"count": 1,

"data": [

"_id": "603d2b2f56b2cf00155b5f5b",

"ticker": "AAPL",

"name": "Apple Inc.",

"currentPrice": 175.50,

"startingPrice": 170.00

]

...

7.3 Transaction Routes (`/api/transactions`)

1. Execute Trade Order (Buy/Sell)

- **HTTP Method:** POST
- **Path:** `/api/transactions`
- **Role Authorized:** Protected (Trader)
- **Request Body:**

```json

"ticker": "AAPL",

"type": "BUY",

"shares": 10

...

- **Response (201 Created):**

```json

"success": true,

"data":

"user": "603d2b2f56b2cf00155b5f5a",

```
"ticker": "AAPL",

"type": "BUY",

"shares": 10,

"price": 175.50,

"timestamp": "2026-06-17T16:00:00.000Z"

...


```

7.4 Admin Routes (`/api/admin`)

1. Get Administrative Statistics

- **HTTP Method:** GET
- **Path:** `/api/admin/stats`
- **Role Authorized:** Protected (Admin)
- **Response (200 OK):**

```
``json

{
  "success": true,
  "data": {
    "users": 10,
    "stocks": 10,
    "transactions": 45
  }
}
```

```

#### #### 2. Create Stock Ticker Listing

- **HTTP Method:** POST
- **Path:** `/api/admin/stocks`
- **Role Authorized:** Protected (Admin)
- **Request Body:**

```json

"ticker": "NVDA",

"name": "NVIDIA Corporation",

"currentPrice": 450.00

```

- **Response (201 Created):**

```json

"success": true,

"message": "Stock ticker NVDA listed successfully"

```

---

## 8. Authentication & Authorization

## Backend Layer

- **JWT Token:** Token signature generated using helper utility `generateToken(userId)` valid for 30 days.
- **Middlewares:**
  - `protect`: Intercepts Authorization headers, decodes JWT, and attaches User model to `req.user`.
  - `authorize('Admin')`: Blocks request access if User role does not match requirements.

## Frontend Layer

- **AuthContext:** Holds state for token and current trader, modifying headers for Axios requests.
  - **ProtectedRoute:** React route wrapper that redirects unauthenticated traders to `/login`.
- 

## 9. User Interface

- **Home:** Main landing page with platform presentation, core features overview, and Call-to-Actions (Login/Register).
  - **Login/Register:** Forms validating password length and inputs with instant trader/admin role toggle fields.
  - **Dashboard (Market):** Live tabular display of stock quotes with tick updates in mint green/soft red colors.
  - **Stock Details:** Chart.js lines visualizer showing history ticks and one-click buying/selling forms.
  - **Portfolio:** Personal summary cards displaying cash balance, asset net worth, shares holdings, and average purchase costs.
  - **Admin Panel:** Statistics panel displaying counts, with forms to add stock tickers and listings controls.
- 

## 10. Testing

### Strategy

- **Unit Testing:** Route controller validation logic tested offline to ensure databases save inputs and bcrypt hashes passwords.
  - **Integration Testing:** Verification of REST API requests and response status codes using Postman collections.
  - **User Acceptance (UAT):** End-to-end user testing verifying order execution flows and theme UI checks.
- 

## 11. Known Issues

- **Concurrency Orders:** Rapid double clicks on buy/sell order buttons within milliseconds can spawn concurrent write requests before Mongoose model validations complete.
-

## 12. Future Enhancements

- **WebSockets Engine:** Implement Socket.io integration to feed live price simulation updates to frontend clients without polling.
- **Advanced Charting:** Integrate advanced TradingView widget charts for detailed technical analysis.